GitHub Workflow Guide for Capstone Teams

QM 2023: Statistics II Spring 2026

Purpose

This guide covers the **GitHub Classroom + Codespaces workflow** for your capstone project. You'll learn how to:

- Work in GitHub Codespaces (cloud-based coding environment)
- Use branches to organize your work
- Commit and push changes
- Collaborate with teammates without conflicts
- Submit milestones through your shared repository

What is GitHub Codespaces?

GitHub Codespaces is a cloud development environment that runs in your web browser. Think of it as VS Code in the cloud—no installation, no setup, just code.

Benefits:

- ☑ Works on any computer (Windows, Mac, Chromebook)
- ☑ Python and all packages already installed
- ☑ Automatically syncs with your team's GitHub repository
- ☑ No "it works on my machine" problems
- ☑ Accessible from anywhere with internet

What you need:

- GitHub account (you should already have one)
- Web browser (Chrome, Firefox, Edge, Safari)
- Internet connection

Part 1: Getting Started with Your Team Repository

Step 1: Accept GitHub Classroom Invitation

REQUIRED: All teams must create their repository through GitHub Classroom.

1. **Find the assignment link** in Blackboard (posted by instructor)
2. **Click the link** → GitHub Classroom will load
3. **First teammate:**
    - Click **Create a new team** or **Join an existing team**
    - If creating: Enter a team name (e.g., `team-reit-wizards`, `crypto-analysts`)
    - **Important:** Use a descriptive, lowercase name with hyphens (no spaces)

4. **Other teammates:**
    - Click the same Classroom link
    - **Join your team** (don't create a new one!)
    - You'll see your team listed—click to join

Result: GitHub creates a private repository named `qm2023-capstone-[team-name]` shared by all teammates.

Step 2: Open Your Repository in GitHub Codespaces

Every teammate does this:

1. Go to **github.com** and sign in
2. Navigate to your team's repository (check your notifications or search for `qm2023-capstone-[team-name]`)
3. Click the green **Code** button
4. Click the **Codespaces** tab
5. Click **Create codespace on main**

What happens:

- GitHub builds a cloud environment (30-60 seconds)
- A VS Code-like editor opens in your browser
- You now have a full Python development environment!

Where are your files?

- In the **Explorer** panel (left sidebar, top icon)
- You'll see folders: `code/`, `data/`, `results/`, etc.

Step 3: Verify Your Environment

In the Codespaces terminal (bottom panel):

```
# Check Python version
python --version
# Should show: Python 3.10 or higher

# Install required packages
pip install -r requirements.txt

# Verify paths
python code/config_paths.py
# Should print: "✓ All paths configured correctly"
```

If terminal isn't visible: Click **View** → **Terminal** (or press Ctrl+ `/` `Cmd+`)

Part 2: Daily Workflow (How to Work on Your Project)

Opening Your Codespace

Every time you start work:

1. Go to **github.com** → your repository
2. Click **Code** → **Codespaces**
3. You'll see your existing Codespace listed
4. Click it to reopen (usually faster than creating new one)

Or: Go directly to github.com/codespaces → see all your Codespaces

---

Making Changes and Committing

Workflow (visual method):

1. **Edit your files** in Codespaces (create/modify Python scripts, notebooks, markdown files)

2. **See what changed:**

   - Click **Source Control** icon (left sidebar, looks like a branching diagram)
   - Changed files appear with "M" (modified) or "U" (untracked/new)

3. **Stage your changes:**

   - Hover over each file → click the **+** icon
   - Or click **+** next to "Changes" to stage all files

4. **Write a commit message:**

   - In the text box above the file list
   - Example: `"Add data cleaning for missing returns"`
   - Be descriptive! Your teammates (and future you) will read this

5. **Commit:**

   - Click the **checkmark** icon (Commit)
   - Or click dropdown → **Commit & Sync** to commit and push in one step

6. **Push to GitHub:**

   - Click **Sync Changes** button
   - Your changes are now saved to the shared repository on GitHub!

Workflow (terminal method):

```
# See what changed
git status

# Stage specific files
git add code/capstone_data_pipeline.py
git add M1_data_quality_report.md
```

```
# Or stage all changes
git add .

# Commit with message
git commit -m "Add data cleaning for missing returns"

# Push to GitHub
git push origin main
```

Getting Your Teammates' Changes

Before you start working each day:

1. **Sync changes:**
   - Click **Source Control** → **Sync Changes** button
   - This pulls the latest version from GitHub

Or via terminal:

```
git pull origin main
```

Why this matters: If your teammate pushed changes last night, you need to pull them before editing. Otherwise, you'll create conflicts.

Part 3: Working with Branches

Why Use Branches?

Scenario: You want to experiment with a new data cleaning approach, but you don't want to break the working code.

Solution: Create a **feature branch** for your experiment.

- **Main branch:** Stable, working code. Always runs without errors.
- **Feature branch:** Experimental work. Safe to break things and try new ideas.

Creating a Feature Branch

Visual method (in Codespaces):

1. Look at the **bottom-left corner** of Codespaces
2. You'll see `main` (current branch)
3. Click `main` → **Create new branch**
4. Enter a name: `m1-data-cleaning` or `test-outlier-removal`
5. Press Enter

Terminal method:

```
git checkout -b m1-data-cleaning
```

Verify: Bottom-left corner now shows `m1-data-cleaning` instead of `main`.

Working on Your Branch

1. **Make changes** to your files (edit, add, delete)
2. **Commit changes** (same process as before—Source Control or `git add` + `git commit`)
3. **Push your branch** to GitHub:

   ```
   git push origin m1-data-cleaning
   ```

Your teammates can see your branch on GitHub but won't be affected unless they switch to it.

Merging Your Branch Back to Main

When your feature is complete and tested:

1. **Switch back to main:**
   - Click bottom-left corner → select `main`
   - Or terminal: `git checkout main`
2. **Pull latest changes from GitHub:**

   ```
   git pull origin main
   ```

3. **Merge your feature branch:**

   ```
   git merge m1-data-cleaning
   ```

4. **Push to GitHub:**

   ```
   git push origin main
   ```

5. **Delete the feature branch (cleanup):**

   ```
   git branch -d m1-data-cleaning
   ```

Result: Your experimental changes are now in `main`, and everyone can see them!

When to Use Branches

Use a branch when:

- Testing a new cleaning method
- Trying a different regression specification
- Experimenting with plot layouts
- Working on a specific milestone section

Stay on main when:

- Making small fixes (typos, comment updates)
- You're confident the change won't break anything

Part 4: Handling Merge Conflicts

What Are Merge Conflicts?

Scenario:

- You edit line 25 of `capstone_data_pipeline.py`
- Your teammate also edits line 25
- Both of you commit and push
- Git doesn't know which version to keep → **conflict!**

How to Resolve Conflicts

When you pull changes and see a conflict:

1. **Git will tell you:**

```
CONFLICT (content): Merge conflict in code/capstone_data_pipeline.py
```

2. **Open the conflicted file** in Codespaces. You'll see:

```
<<<<<<< HEAD
# Your version
df = df.dropna(subset=['ret'])
=======
# Teammate's version
df = df.dropna(subset=['ret', 'mcap'])
>>>>>>> main
```

3. **Decide which to keep:**
    - **Keep yours:** Delete teammate's version and markers
    - **Keep theirs:** Delete your version and markers
    - **Combine both:** Write a new version that incorporates both changes
4. **Example resolution (combined):**

```
# Combined version (keep both columns)
df = df.dropna(subset=['ret', 'mcap'])
```

5. **Remove the conflict markers:**
    - Delete `<<<<<<<`, `=======`, `>>>>>>>`
    - Save the file
6. **Commit the resolution:**

```
git add code/capstone_data_pipeline.py
git commit -m "Resolve merge conflict: keep both ret and mcap in dropna"
git push origin main
```

Preventing Conflicts

Best practices:

- **Communicate:** "I'm editing the data cleaning section today"
- **Pull often:** Start each work session with `git pull`
- **Divide files:** One person owns M1 pipeline, another owns M2 notebook
- **Use branches:** Work on separate branches, merge carefully
- **Commit frequently:** Small commits = smaller conflicts

Part 5: Team Collaboration Patterns

Pattern 1: Divide and Conquer (Parallel Work)

Best for: Different files or independent sections

Example (M1 Data Pipeline):

- **Sarah:** Works on `code/load_reit_data.py` (loads primary dataset)
- **Alex:** Works on `code/fetch_fred_data.py` (fetches FRED indicators)
- **Jordan:** Works on `code/merge_data.py` (combines datasets)
- **Taylor:** Works on `M1_data_quality_report.md` (documentation)

Why it works: Everyone edits different files → no conflicts!

Workflow:

1. Each person creates their own branch
2. Work independently
3. Push branches to GitHub
4. One person (Release Manager) merges all branches to main
5. Test that the full pipeline runs

Pattern 2: Sequential Handoffs

Best for: Work that depends on previous steps

Example (M2 EDA Dashboard):

1. **Week 1:** Sarah creates correlation heatmap → commits to main
2. **Week 2:** Alex builds on Sarah's work → adds lagged analysis
3. **Week 3:** Jordan adds sector comparisons
4. **Week 4:** Taylor integrates all plots into final notebook

Workflow:

- Each person pulls latest `main` before starting
- Works on their section
- Commits and pushes when done
- Next person pulls and continues

Pattern 3: Pair Programming

Best for: Complex tasks, learning together

Example:

- Two teammates open the same Codespace (each in their own browser)
- One person "drives" (types code)
- Other person "navigates" (suggests logic, catches errors)
- Commit together: `git commit -m "Pair programming: Sarah + Alex - FRED integration"`

Why it works: Real-time collaboration, knowledge sharing, fewer bugs

---

Part 6: Milestone Submission Workflow

Final Checklist Before Submission

One week before deadline:

- ☐ All code is in `main` branch
- ☐ All feature branches merged

- ☐ Code runs without errors (test in fresh Codespace)
- ☐ All deliverables present:
  - M1: `capstone_data_pipeline.py`, `*_analysis_panel.csv`, `*_metadata.json`, `M1_data_quality_report.md`, `AI_AUDIT_APPENDIX.md`
  - M2: Jupyter notebook with 8+ visualizations, hypothesis section
  - M3: Regression script, tables, diagnostics
  - M4: Final memo (PDF or MD), individual addenda
- ☐ Files in correct folders (`code/`, `data/final/`, `results/`)
- ☐ No hardcoded paths (`C:\Users\...`)
- ☐ No large files committed (>10 MB)

Submission Process

Assign a "Release Manager" per milestone (rotate roles):

1. **Release Manager duties:**
   - Pull latest main: `git pull origin main`
   - Run all scripts to verify they work
   - Check file organization
   - Tag the submission:

     ```
     git tag -a M1-submission -m "Milestone 1: Data Pipeline complete"
     git push origin M1-submission
     ```

2. **Communicate with team:**
   - "M1 submission tagged and pushed. Everyone verify by opening a fresh Codespace and running the pipeline."
3. **All teammates verify:**
   - Open a new Codespace (or close and reopen existing one)
   - Pull latest: `git pull origin main`
   - Run the submission files (e.g., `python code/capstone_data_pipeline.py`)
   - Confirm: "Works for me! ✓"
4. **Notify instructor:**
   - Submit the GitHub repository URL in Blackboard (if required)
   - Or instructor auto-grades from GitHub Classroom

After Submission: Start Next Milestone

Don't wait! Start next milestone immediately:

```
# Create new branch for next milestone
git checkout -b m2-eda-dashboard

# Begin work
# Commit and push as you go
```

Why: Each milestone builds on the previous one. Starting early gives you time to iterate.

Part 7: Common Issues and Solutions

Issue 1: "I accidentally committed a large data file"

Problem: You committed `data/raw/reit_master_raw.csv` (50 MB), and now pushes are slow or failing.

Solution:

```
# Remove from Git (but keep local copy)
git rm --cached data/raw/reit_master_raw.csv

# Add to .gitignore
echo "data/raw/*.csv" >> .gitignore

# Commit changes
git add .gitignore
git commit -m "Remove large data file from Git tracking"
git push origin main
```

Prevention: Always check `.gitignore` before committing large files.

Issue 2: "My Codespace is slow or frozen"

Solutions:

1. **Refresh browser:** Sometimes Codespaces needs a reload
2. **Stop and restart Codespace:**
   - Go to github.com/codespaces
   - Click ··· → **Stop codespace**
   - Wait 10 seconds → **Restart**
3. **Delete and create new Codespace:**
   - Stop current Codespace
   - Delete it (your code is safe on GitHub!)
   - Create new Codespace from main branch
   - Pull latest changes: `git pull origin main`

Issue 3: "I can't see my teammate's changes"

Checklist:

- ☐ Did they commit? Check GitHub repository → **Commits** tab
- ☐ Did they push? Look for their commit in GitHub web interface
- ☐ Did you pull? Run `git pull origin main` in your Codespace

Still not seeing changes?

- Verify you're on the same branch (bottom-left corner)
- Check if they pushed to a different branch

Issue 4: "Git says 'divergent branches'"

What happened: You and a teammate both made commits without pulling first.

Solution:

```
# Pull and merge
git pull origin main

# Resolve conflicts if any (see Part 4)

# Push your merged version
git push origin main
```

Issue 5: "I deleted a file by accident"

Don't panic! Git saves everything.

If you haven't committed the deletion:

```
git checkout -- path/to/deleted_file.py
```

If you already committed:

```
# Find the commit hash where file existed
git log --oneline

# Restore file from previous commit
git checkout <commit-hash> -- path/to/deleted_file.py

# Commit the restoration
git add path/to/deleted_file.py
git commit -m "Restore accidentally deleted file"
git push origin main
```

Part 8: GitHub Best Practices for Teams

Commit Messages: Do's and Don'ts

Good commit messages:

- `"Add FRED API integration for FEDFUNDS and MORTGAGE30US"`
- `"Fix missing value handling: drop rows with null returns"`
- `"M2: Create correlation heatmap for outcome vs drivers"`
- `"Update README with team member names and research question"`

Bad commit messages:

- `"fixed stuff"` ← What stuff?
- `"updates"` ← What updates?
- `"asdfasdf"` ← Not helpful
- `"final version"` ← Which final? There will be many!

Format: Present tense, descriptive, 50 characters or less. If you need more detail, add a blank line and explanation.

Pull Request Workflow (Optional but Recommended)

For major milestones: Instead of merging branches directly, use **Pull Requests** for code review.

Workflow:

1. Push your feature branch to GitHub: `git push origin m1-data-cleaning`
2. Go to GitHub repository → **Pull requests** → **New pull request**
3. Select: `base: main` ← `compare: m1-data-cleaning`
4. Click **Create pull request**
5. Add description: "This PR adds data cleaning logic for M1 pipeline"
6. Assign a teammate as reviewer
7. Reviewer checks code, adds comments, approves
8. Click **Merge pull request**

Why: Formal code review catches bugs, ensures quality, documents decisions.

Communication Norms

Use GitHub Issues for task tracking:

1. Go to repository → **Issues** → **New issue**
2. Title: `"M1: Fetch FRED data via pandas-datareader"`
3. Assign to teammate
4. Label: `milestone-1`, `enhancement`
5. When done, close issue with a commit: `git commit -m "Fetch FRED data (closes #3)"`

GitHub tracks:

- Who did what (commit history)
- When changes were made (timestamps)
- Why changes were made (commit messages, PR descriptions)

Part 9: Contribution Tracking

Why Track Contributions?

Required for M4 individual addendum:

- You must document your personal contributions
- Peer evaluations (Week 15) assess individual effort
- GitHub commit history is visible to instructor

Simple Contribution Log

Create `team_contributions.md` in your repository:

```
# Team Contribution Log

## Week 4: M1 Planning (Feb 9-13)
- **Sarah:** Set up GitHub repo, created folder structure, drafted README (3 hours)
- **Alex:** Downloaded REIT data, explored with pandas, identified missing values (2.5 hours)
- **Jordan:** Researched FRED API, tested pandas-datareader, documented examples (2 hours)
- **Taylor:** Outlined M1 cleaning decisions, started data quality report template (1.5 hours)

## Weeks 5-6: M1 Execution (Feb 16-25)
- **Sarah:** Wrote data cleaning section (missing values, outliers, duplicates) (4 hours)
- **Alex:** Integrated FRED API, merged with REIT data, validated merge (3 hours)
- **Jordan:** Created metadata JSON, verified final panel structure (1.5 hours)
- **Taylor:** Wrote M1_data_quality_report.md, final testing, AI Audit (3 hours)
```

Update weekly. Each teammate adds their hours and tasks.

GitHub Contribution Graph

Instructor can see:

- Number of commits per person
- Lines of code added/removed
- Files edited

- Commit timestamps

Tip: Commit frequently throughout the week (not just Sunday night before deadline). Shows consistent effort.

Part 10: Workspace Management

Codespaces Limits

GitHub Free accounts:

- 60 hours/month of Codespaces usage (core instances)
- 15 GB storage per month

For this course:

- Usually sufficient (15 hours/week per student if all work done in Codespaces)
- If you hit limits, work can continue locally (but Codespaces is strongly recommended)

Stopping Your Codespace (Save Resources)

When you're done working:

1. Commit and push your changes
2. Close the browser tab
3. Codespace auto-stops after 30 minutes of inactivity

Or manually stop:

- Go to github.com/codespaces
- Click ··· next to your Codespace → **Stop codespace**

Why: Saves your free usage hours for when you actually need them.

Multiple Codespaces

Can you have multiple Codespaces?

- Yes! You can create separate Codespaces for different branches
- Example: One Codespace on `main`, another on `m2-eda-dashboard`

When to use:

- Testing experimental code without affecting stable main branch
- Working on two milestones in parallel

Cleanup: Delete old Codespaces when you're done (github.com/codespaces → Delete).

Summary: Your GitHub Workflow Checklist

Daily workflow:

- ☐ Open Codespace from GitHub
- ☐ Sync changes: `git pull origin main`
- ☐ Create branch if experimenting: `git checkout -b feature-name`
- ☐ Make changes, test code
- ☐ Commit: Stage files → Write message → Commit
- ☐ Push: `git push origin main` (or your branch)
- ☐ Stop Codespace when done (saves resources)

Weekly workflow:

- ☐ Team meeting: Assign tasks, update contribution log
- ☐ Pull latest changes before starting work
- ☐ Commit small, logical changes (not one giant commit)
- ☐ Sync changes daily (avoid merge conflicts)
- ☐ Code review before merging branches

Milestone workflow:

- ☐ One week before: Verify all code runs, all files present
- ☐ Assign Release Manager for final check
- ☐ Tag submission: `git tag -a M1-submission -m "Milestone 1 complete"`
- ☐ All teammates verify by testing in fresh Codespace
- ☐ Submit repository URL in Blackboard (if required)
- ☐ Start next milestone immediately

Resources

- **GitHub Docs:** docs.github.com/codespaces
- **Git Cheat Sheet:** education.github.com/git-cheat-sheet
- **Office Hours:** Dr. Seagraves, Mon & Wed 3-5 PM (Helm 122-D)
- **Course GitHub Discussions:** Post questions, help teammates

---

FAQ

Q: Do I need to install Git on my computer? A: No! GitHub Codespaces has Git built-in. Everything runs in the browser.

Q: Can I work offline? A: No, Codespaces requires internet. But you can download files and work locally if needed (not recommended).

Q: What if I'm more comfortable with local development (VS Code on my laptop)? A: You can clone the repository locally and work that way, but you'll need to manage Python installation, virtual environments, and package dependencies yourself. Codespaces is strongly recommended for consistency.

Q: Can I use Jupyter notebooks in Codespaces? A: Yes! Create `.ipynb` files, and Codespaces opens them in an interactive notebook interface.

Q: What happens if two people edit the same file at the same time? A: Git will detect the conflict when you try to push/pull. Follow the merge conflict resolution steps (Part 4).

Q: How do I know who made which changes? A: Use `git log` or click a file in GitHub → **Blame** view → see line-by-line author history.

Q: Can I revert to a previous version of a file? A: Yes! Find the commit hash: `git log --oneline` → Restore file: `git checkout <hash> -- path/to/file.py`

You're now ready to collaborate like professional data scientists!

Document prepared for QM 2023: Statistics II, Spring 2026, University of Tulsa Contact: Dr. Cayman Seagraves (cayman-seagraves@utulsa.edu)